

MATH19622-1E2: Introduction to Probability and Statistics

4 Lecture Series (90 minutes per Lecture)

Anthony R. Thornton

2025/2026

Contents

I.1 About the course	3
I.1.1 History of the course	3
I.1.2 Additional reading	4
I.1.2.1 Primary Resources	4
I.1.2.2 Supplementary / General Reference	4
I.1.2.3 For the Mathematically Interested	4
I.1.3 Course Structure	4
I.1.4 Assessment	5
I.1.5 Course Overview	5
I.1.5.1 GitHub Repository	5
1 Descriptive Statistics	5
1.1 What is Descriptive Statistics?	6
1.1.0.1 The Goal	6
1.2 Population vs Sample	6
1.2.1 Why do we sample in Engineering?	7
1.2.2 Representative Sample	7
1.2.2.1 The Pitfall: Bias	8
1.2.2.2 Discussion: Is this Representative?	8
1.2.3 The Problem of ‘Luck’ (Sampling Error)	8
1.2.3.1 The Solution: Confidence Intervals	9
1.2.3.2 Key Property: Sample Size	9
1.2.4 Ethics in Statistics: The ‘P-Hacking’ Problem	9
1.2.4.1 This is P-Hacking (Data Dredging)	10
1.3 Types of Data	10
1.3.1 Sources of data	10
1.3.1.1 Finite Population	10
1.3.1.2 Ongoing Process	11
1.3.2 Classifying Data	11
1.3.2.1 Discrete vs. Continuous Data	11

1.4 Measures of Central Tendency	12
1.4.1 Arithmetic Mean	12
1.4.2 Median	12
1.4.3 Mode	13
1.4.4 Worked Example	13
1.4.4.1 Key Takeaway	14
1.4.5 Why does this happen? The Physics of the Mean	14
1.5 Python to the Rescue	14
1.5.1 Setting up VS code for python	14
1.5.1.1 The Engineering Stack	14
1.5.2 Context: The ‘SciPy Stack’	15
1.5.3 The Key Components	15
1.5.4 Our Interactive Mean versus Median Demo	17
1.6 Summary so far	19
1.7 Measures of Dispersion (Spread)	19
1.7.1 Why Central Tendency isn’t enough	19
1.7.2 Why do we measure Spread?	20
1.7.2.1 Real-World Engineering Constraints	20
1.7.3 The Range	20
1.7.4 Interquartile Range (IQR)	21
1.7.4.1 The IQR Formula	21
1.7.4.2 Visualising the IQR	22
1.7.5 The Variance	22
1.7.5.1 Population vs Sample	22
1.7.5.2 Bessel’s Correction ($n - 1$)	22
1.7.5.3 Standard Deviation	23
1.7.6 Worked example	24
1.7.7 Python Implementation	24
1.7.8 Variance vs Standard Deviation	25
1.7.8.1 Variance (σ^2)	25
1.7.8.2 Standard Deviation (σ)	25
1.8 Summary: Descriptive Statistics	25
1.8.0.1 Location (The Signal)	25
1.8.0.2 Dispersion (The Noise)	25
1.9 Exercise Questions	26
 2 Probability Fundamentals	 27
2.1 Outline of next block	27
2.2 Probability Fundamentals	28
2.2.1 Definitions & Notation	28
2.2.2 The Addition Law	29
2.2.2.1 For Mutually Exclusive Events	29
2.2.2.2 The General Addition Rule	29
2.2.3 The Multiplication Law	31
2.2.3.1 The Multiplication Law (Intersections)	31

2.2.4 Summary	33
2.2.4.1 Next Up: Discrete Probability Distributions	33
2.3 Exercise Questions	33
3 Discrete Probability Distributions	35
3.1 Introduction	35
3.2 The Binomial Distribution	36
3.2.0.1 Conditions for a Binomial Model	36
3.2.0.2 Binomial Shortcuts	37
3.3 The Poisson Distribution	38
3.3.0.1 When to use Poisson in EEE:	38
3.3.0.2 Poisson Shortcuts	39
3.4 Summary: Discrete Distributions	40
3.4.0.1 Next Up: Continuous Probability Distributions	41
3.5 Exercise Questions	41

I.1 About the course

I.1.1 History of the course

This course was *newly* developed by **Anthony R. Thornton** for the 2025/2026 academic year.
It is based on:

Mathematics 2M1 — Probability and Statistics written by Robert Gaunt, School of Natural Sciences, University of Manchester, and David J. Silvester, School of Natural Sciences, University of Manchester.

Special thanks to Google Gemini for proofreading and structural suggestions.

I.1.2 Additional reading

It should be noted that this reader, with associated lecture slides forms the *entire* content of the course and is complete. However for the interested student a selection of text books is included some following the same style as the course and other that are more mathematically rigorous. It should be noted all the resources listed below are full **open access**.

I.1.2.1 Primary Resources

- **Module 1 (Descriptive Statistics):** *Introductory Statistics* by **OpenStax**. <https://openstax.org/details/books/introductory-statistics>
- **Modules 2–4 (Probability):** *Introduction to Probability, Statistics, and Random Processes* by **H. Pishro-Nik**. (Tailored for EEE). <https://www.probabilitycourse.com>

I.1.2.2 Supplementary / General Reference

- *OpenIntro Statistics* by **Diez, Çetinkaya-Rundel, & Barr**. (A clean, concise alternative for general statistics concepts). <https://www.openintro.org/book/os/>
- *Think Stats* by **Allen B. Downey**. (Teaches statistics through Python programming). <https://greenteapress.com/wp/think-stats-2e/>
- **Khan Academy (Statistics & Probability)**. (Video tutorials useful for visualising histograms and distributions). <https://www.khanacademy.org/math/statistics-probability>

I.1.2.3 For the Mathematically Interested

Introduction to Probability by **Grinstead & Snell**. (Classic text with rigorous proofs and derivations). <https://math.dartmouth.edu/~prob/prob/prob.pdf>

I.1.3 Course Structure

This course is designed to cover the final part of Mathematics for EEE 1E2, namely:

Statistics Discrete and continuous data. Mean, mode, and median. Range, variance and **standard deviation**. Frequency curves: Normal distribution and standardised normal.

& Probability Empirical and classical. Addition and multiplication laws. Probability distributions. Mean and standard deviation. Binomial and Poisson distribution. Normal distribution curves.

It will consist of four approximately 90-minute lectures. The course is written in a highly practical way, focusing on the application of the topic rather than proofs, derivations, and strict mathematical rigour. However, it also teaches you how to identify the underlying mathematics in real-world scenarios, using many examples drawn from electrical engineering. Ultimately, this is not a pure maths course; it is a maths *for* engineering course.

The first two elements: Differential Vector Calculus and Integral Vector Calculus are **covered** by a separate course taught by *Igor Chernyavsky*.

I.1.4 Assessment

The assessment for these elements of the course consists of:

- **Online Coursework** (Week 10) Contributes **10%** to the final grade.
- **End-of-Semester Examination** Contributes **80%** to the final grade.

The remaining **10%** is assessed in the section taught by **Igor Chernyavsky**. Note both these tests will also include questions from the earlier parts 1 and 2.

I.1.5 Course Overview

Overview: 5 Thematic Blocks delivered across 4 Lectures (100 mins each).

Session	Thematic Blocks	Learning Outcomes
Lecture 1	Overview of Part Block 1: Descriptive Statistics	ILO 5: Data Analysis
Lecture 2	Block 2: Prob. Fundamentals Block 3: Discrete Distributions	ILO 6: Modeling
Lecture 3	Block 4: Continuous Distributions	ILO 6: Problem Solving
Lecture 4	Block 5: Linear Regression	ILO 5: Interpretation

While official PDFs are on **Canvas**, the ‘Living’ version of this course (including live Python code and LaTeX source) is hosted on GitHub.

I.1.5.1 GitHub Repository

<https://github.com/antrthorn/UoM-MATH19622-Public>

How to get the code (for the interested):

1. Clone the Repo:

```
1 git clone https://github.com/antrthorn/UoM-MATH19622-Public.git
2
```

2. Run the Examples:

```
1 uv run path/to/script.py
2
```

Note: Use the VS Code Dev Container for a zero-install Python environment.

Block 1

Descriptive Statistics

1.1 What is Descriptive Statistics?

Welcome to the first module of this course: Descriptive Statistics. As Electrical Engineers, you deal with **signals** and **systems**. Up until now, much of your engineering education has likely lived in the ‘Ideal World’. In that world, a $100\ \Omega$ resistor provides exactly $100\ \Omega$ of resistance, a 5V source delivers exactly 5.000V, and Ohm’s Law ($V = IR$) works with absolute, deterministic perfection.

Unfortunately, the real world is messy. When you open a batch of physical resistors, you will find they actually measure $98\ \Omega$, $101\ \Omega$, or $103\ \Omega$ due to manufacturing tolerances. When you measure a voltage across a sensor, thermal fluctuations will cause the reading to jitter. In electrical engineering, this uncertainty is not a mistake—it is an unavoidable physical reality.

This module is about learning the mathematical vocabulary to describe that reality. Descriptive statistics provides us with the tools to take large, messy sets of real-world data and extract meaningful engineering insights.

Over the next few pages, we will cover two main concepts:

- **Measures of Central Tendency:** How to find the ‘Signal’ hiding in the data (using the Mean, Median, and Mode).
- **Measures of Dispersion:** How to quantify the ‘Noise’ or uncertainty around that signal (using Variance and Standard Deviation).

By the end of this section, you will not just be able to calculate an average; you will understand how to use these metrics to assess component reliability, evaluate system tolerances, and make informed engineering decisions. We will also introduce standard Python libraries (NumPy and SciPy) so you can handle large datasets just as you would in industry.

To illustrate the point:

- **The Ideal World:** $V = IR$. A perfect resistor has exactly $100\ \Omega$ resistance.
- **The Real World:** A resistor has a tolerance (e.g., $100 \pm 5\ \Omega$). Thermal noise introduces random fluctuations in voltage.

1.1.0.1 The Goal

Statistics allows us to quantify this uncertainty, distinguish the **signal** from the **noise**, and build reliable systems from imperfect components.

1.2 Population vs Sample

It is crucial to distinguish between **parameters**, which describe an entire *population*, and **statistics**, which describe a *sample* taken from that population.

Definition: Population

The entire set of items/individuals of interest.

- Example: *Every* capacitor produced by a factory in 2025.
- Measures are called **Parameters** (e.g., population mean, μ).

Definition

$\mathcal{S} \subseteq \mathcal{P}$ [Sample] A subset of the population selected for analysis.

- Example: A random batch of 50 capacitors tested for failure.
- Measures are called **Statistics** (e.g., sample mean, $\bar{x}$).

Example: If we are interested in the average height of all students at the University of Manchester (the *parameter*), we might statistically estimate this by measuring a *sample* of students sitting in The Place at 12:00 on a Monday.

1.2.1 Why do we sample in Engineering?

In pure mathematics, it is sometimes possible to measure an entire population. In electrical engineering, this is often impossible due to time, cost, or the nature of the test. If a quality control test is **destructive** (for example, applying overvoltage to a capacitor until it bursts to find its ultimate failure point), testing the entire population would leave you with zero product to sell! Therefore, engineers are forced to test a small, representative sample and use statistics to infer the safety parameters of the remaining batch. *In this course, we mostly calculate Statistics to estimate Parameters.*

1.2.2 Representative Sample

To draw valid conclusions about a population, our sample must be fair. By fair we mean not subject to any bias in the way the data is collected. The formulas we use in statistics carry a dangerous underlying assumption: they blindly assume that your sample perfectly reflects the wider population. If your sample does not, every mean, variance, and standard deviation you calculate will be mathematically flawless, but practically useless.

Definition: Representative Sample

A *representative sample* is a subset that accurately reflects the characteristics of the larger **population**.

The Goal: Randomness To be representative, samples are usually collected randomly.

Definition: Simple Random Sample

Every member of the population has an **equal chance** of being selected.

To achieve this we have to remove human choice and systematic errors. In casual English, ‘random’ often just means ‘haphazard’. If a manager tells an engineer to ‘go to the warehouse and pick 50 random power supplies for testing’, the engineer will walk in, go to the nearest pallet, and pull 50 boxes from eye level. This is **not** a Simple Random Sample. The boxes at the back of the warehouse, or on the top shelf, had a 0% chance of being selected. If a forklift damaged the boxes near the door, your sample is now heavily biased. Creating a true Simple Random Sample (SRS) in an engineering environment requires removing human choice entirely. It is a strict, three-step process:

Recipe (Generating a Simple Random Sample)

1. **Identify and Enumerate:** Define the absolute boundary of your population (e.g., a specific batch of 5,000 microcontrollers) and ensure every single unit has a unique identifier, such as a serial number from 1 to N .
2. **Algorithmic Selection:** Do not pick items manually. Instead, use a computerized Random Number Generator (such as `numpy.random.choice()` in Python) to draw your target sample size (n) from the pool of available IDs.
3. **Blind Retrieval:** The engineer must physically extract those *exact* serial numbers for the test bench. You must explicitly ignore how physically convenient or inconvenient they are to reach in the warehouse.

This algorithmic approach is the only physical way to guarantee that every component has an exactly equal and independent chance of being tested, thereby protecting your statistics from human bias.

1.2.2.1 The Pitfall: Bias

Bias occurs when the sample systematically favours certain outcomes. This will mean certain outcomes are more likely than others, destroying the assumption of randomness. For example, suppose a quality assurance team decides to test the reliability of microchips. For convenience, they test only the first 100 chips produced right after the machines are turned on at 8:00 AM. The issues with this is that this sample completely ignores the reality of the manufacturing floor. Machines heat up, calibration drifts, and operators experience fatigue as the day progresses. By only sampling the ‘fresh’ morning batch, the engineers are systematically ignoring defects

caused by machine overheating at 4:00 PM. The Mathematical Consequence of this bias, is that any statistics calculated from this sample (like the sample mean failure rate, $\bar{x}$) will artificially underestimate the true population parameter (μ). You can execute the mathematics perfectly in Python, but because the sample was biased, the final engineering conclusion will be dangerously wrong.

1.2.2.2 Discussion: Is this Representative?

Example 1 (The Solar Engineering). Scenario: An engineering team is designing a new solar power inverter. To test its reliability, they install 50 prototype units on rooftops in **Manchester, UK** and monitor them for a year.

The data shows a 99.8% reliability rate. The team concludes the inverter is ready for a global rollout, including markets in **Dubai** and **Arizona**.

Question: Is this sample representative of the global operating environment?

- Why or why not?
- What variables might be missing from the sample?

Solution 2 (The Problem: Environmental Bias). **No.** *The sample is biased towards a cool, cloudy climate.*

- **Missing Variable:** *Extreme heat ($> 40^{\circ}\text{C}$) effectively derates electronics and stresses cooling systems.*
- **Consequence:** *High failure rates possible in hot climates.*

1.2.3 The Problem of ‘Luck’ (Sampling Error)

Even if our sample is perfectly random and representative (no bias), it is still a subset. By pure chance, we might pick slightly more ‘high’ values than ‘low’ values.

Definition: Sampling Variation (Random Error)

Random error is the difference between the sample statistic ($\bar{x}$) and the true **population parameter** (μ) caused purely by chance.

Example 3. You flip a fair coin 10 times. You *should* get 5 Heads.

- **Result:** You get 8 Heads.
- **Is the coin biased?** Probably not. You just had ‘bad luck’ (random variation) in your small sample.

If you were to flip the coin 100 or 1,000 times, would you expect the sample mean ($\bar{x}$) to approach the true population mean (μ)? The answer is yes. In fact, the standard error between $\bar{x}$ and μ shrinks proportionally to $1/\sqrt{n}$, where n is your sample size. This is a fundamental concept known as the **Law of Large Numbers**.

In the Probability section later in this course, we will return to this exact scenario and mathematically calculate the precise chance of getting 8 heads from 10 fair coin flips.

So, how do we handle this unavoidable uncertainty when reporting our statistics to other engineers? Instead of providing a single, fragile number, we provide a range. This naturally brings us to our next topic: Confidence Intervals.

1.2.3.1 The Solution: Confidence Intervals

When you hand a datasheet to another engineer, providing a single point estimate (like $\bar{x} = 5.0\text{V}$) is dangerous because it hides the uncertainty of your small sample. Instead, we construct a **Confidence Interval (CI)**.

A CI provides a range of plausible values for the true population parameter, explicitly communicating the margin of error.

Because of this variation, we rarely state a single number. Instead, we calculate a range:

‘The voltage is $5.0\text{V} \pm 0.1\text{V}$ with 95% confidence.’

What does 95% confidence mean? It means that if we were to repeat our sampling process 100 times, we would expect 95 of those calculated intervals to successfully capture the true, underlying population average.

1.2.3.2 Key Property: Sample Size

There is a fundamental difference between Bias and Random Error. If your sampling method is biased (e.g., testing only the top shelf of the warehouse), testing 10,000 units instead of 10 will not help you; it will just give you a very precise, entirely wrong number.

However, if your sample is truly random, you can shrink your random error simply by testing more units. As sample size (n) increases, the Confidence Interval becomes narrower. In engineering design, this creates a direct, real-world trade-off: the statistical precision of your data versus the financial cost of testing more components.

1.2.4 Ethics in Statistics: The ‘P-Hacking’ Problem

Since ‘bad luck’ (random variation) happens, can we exploit it?

We just established that random variation (sampling error) is an unavoidable physical reality. Because this variation exists, a dangerous question arises: if we test something enough times, can we weaponise that randomness to get the result we want?

Example 4 (The (Un)ethical Engineer). An engineer wants to prove a new battery lasts longer.

- They run **100 separate small experiments**.
- In 95 cases, the new battery is no better than the old one.
- In 5 cases, by pure random chance, the new battery performs amazingly.
- **Action:** They publish *only* the 5 ‘successful’ results and hide the rest.

1.2.4.1 This is P-Hacking (Data Dredging)

The Math Behind the Fraud: Why did 5 batteries perform amazingly? Recall the concept of 95% confidence. If you test a completely average, unimproved battery 100 times, statistical theory dictates that around 5% of those tests will yield extreme outlier results purely by chance. The engineer hasn’t invented a better battery; they have simply captured 5 instances of positive statistical noise.

The Ethical Breach: This practice is known as **p-hacking** or **data dredging**. The most insidious part of this fraud is that the data in those 5 reports isn’t technically ‘fake’—the multimeter really did read those high values. However, by intentionally omitting the 95 failed tests, the engineer has completely distorted the reality of the component.

The Engineering Consequence: If a company bases a product rollout on those 5 cherry-picked reports, the batteries will immediately revert to their true mean in the real world. If those batteries are in consumer electronics, it causes mass recalls. If they are in medical devices or aerospace systems, it costs lives.

The Golden Rule of Statistical Ethics: You must report **all** experimental attempts and data points, not just the ones that fit your desired conclusion or make your design look good.

1.3 Types of Data

We have so far discussed the difference between statistics of a sample and parameters of the population. Next, we will briefly discuss the sources of engineering data and how we classify it.

Understanding your data type is the crucial first step before writing any Python code or applying any formulas. Applying the wrong statistical tool to the wrong data type will yield mathematically flawless but practically nonsensical results.

1.3.1 Sources of data

Definition: Data

Data is the set of observations or measurements we collect.

In Engineering, it comes from two main sources:

1.3.1.1 Finite Population

This represents a physical, countable group of existing items. In EEE, this usually relates to **Quality Assurance (QA)** and hardware manufacturing. For example, characterising a specific batch of 500 resistors to ensure they meet a 5% tolerance threshold. The goal is to evaluate the present state of this specific batch.

1.3.1.2 Ongoing Process

This represents a system that generates an infinite stream of data over time. In EEE, this relates to **Signal Processing** and system monitoring. For example, measuring the thermal noise on a communication channel. The goal here is not just to characterise the past, but to model the process so you can predict and filter out future noise.

*In both cases, we use **samples** (small datasets) to estimate the properties of the whole.*

1.3.2 Classifying Data

Data generally falls into two broad categories:

1. **Qualitative (Categorical):** Non-numerical descriptions.
 - Examples: Component colour codes, Pass/Fail status.
2. **Quantitative (Numerical):** Measured or counted values.
 - This is our main focus in EEE.
 - Further split into **Discrete** and **Continuous**.

Why does this distinction matter? Because mathematics is blind. A computer will happily let you assign the number ‘1’ to a Red wire and ‘2’ to a Blue wire, and then use `numpy.mean()` to tell you that the ‘average’ wire is 1.5. This is mathematically valid but practically absurd. Qualitative data requires you to count frequencies (the Mode), while Quantitative data allows you to perform true arithmetic (the Mean and Variance). These ideas of mean, mode and variance will be introduced in the next section.

1.3.2.1 Discrete vs. Continuous Data

As stated above, quantitative data can be either *discrete* or *continuous*. Below we summarise the difference.

1. Discrete Data

- Countable values (integers).
- No intermediate values exist.
- **EEE Examples:**
 - Number of packets lost in transmission.
 - Number of electrons (at quantum scale).
 - Digital logic levels (0 or 1).

2. Continuous Data

- Measurable values (real numbers).
- Can take any value within a range.
- **EEE Examples:**
 - Voltage across a capacitor.
 - Current in a wire.
 - Signal propagation delay.

The Digital Illusion (A Warning for EEES) As electrical engineers working with microcontrollers, you must be careful not to confuse physical reality with digital representation. A voltage across a capacitor is a **continuous** physical property—it smoothly transitions through infinite microscopic values as it charges. However, when you measure that voltage using an Analog-to-Digital Converter (ADC) on an Arduino or Raspberry Pi, the ADC ‘quantises’ that voltage into discrete digital steps (e.g., integers from 0 to 1023 for a 10-bit ADC). Therefore, while the physical signal in the copper trace is continuous, the dataset you end up analysing in Python is technically **discrete**.

1.4 Measures of Central Tendency

So far, we have explained the difference between a sample and an entire population, and the types of data we will encounter. Next, we consider how to mathematically describe and compare different datasets.

We often want to summarise a dataset with a single value that represents the **centre** or **location** of the data.

The three most common measures are:

1. **Mean:** The arithmetic average.

2. **Median:** The middle value.
3. **Mode:** The most frequent value.

1.4.1 Arithmetic Mean

The mean is the most common measure of centre. It is the sum of all values divided by the number of observations.

Definition: Sample Mean ($\bar{x}$)

For a sample of size n with values $x_1, x_2, \dots, x_n$:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

Recipe (Calculating the Mean)

1. Sum all the values.
2. Divide by the number of values.

Note:

- For a **Population**, we use the symbol μ .
- The mean is highly sensitive to **outliers** (extreme values).

1.4.2 Median

Definition: Median

The *median* is the middle value when the data is ordered from smallest to largest.

Recipe (Calculating the Median)

1. Sort the data: $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$.
2. Find the position:
 - If n is **odd**: The median is the exact middle number.
 - If n is **even**: The median is the mean of the two middle numbers.

The median is **robust** (less affected by outliers).

1.4.3 Mode

Definition: Mode

The *mode* is the value that occurs most frequently in the dataset.

Recipe (Calculating the Mode)

1. List all unique values in the dataset.
2. Count the frequency (number of occurrences) for each value.

3. Identify the value(s) with the highest frequency.
 - If multiple values share the highest count, the data is **multimodal**.
 - If all values appear only once, there is **no mode**.
- Useful for Qualitative data (e.g., ‘The most common failure mode is overheating’).

1.4.4 Worked Example

Example 5 (Network Router Latency). An engineer measures the latency (in ms) of 5 packets passing through a network router:

Data: 12, 15, 12, 200, 14

Questions:

1. Calculate the mean, median, and mode.
2. Draw an engineering conclusion: Which measure best represents the network’s typical performance?

Solution 6. *Part 1: Calculations*

- **Mean:**

$$\bar{x} = \frac{12 + 15 + 12 + 200 + 14}{5} = \frac{253}{5} = \mathbf{50.6 \text{ ms}}$$

- **Median:** Sort the data: 12, 12, **14**, 15, 200.

$$\text{Median} = \mathbf{14 \text{ ms}}$$

- **Mode:** The most frequent value is **12 ms**.

. *Part 2: Interpretation*

- **Mean (50.6 ms):** Heavily skewed by the single outlier (200 ms). This metric falsely suggests the entire network is slow, which is only true for 1 packet (perhaps due to a temporary buffer spike).
- **Median (14 ms):** Gives a much more accurate representation of the ‘typical’ performance the user will experience.

1.4.4.1 Key Takeaway

Always look at the data distribution. If the data is skewed (like network latency, income, or component lifespans often are), the **median** is a much safer measure of the centre than the **mean**.

1.4.5 Why does this happen? The Physics of the Mean

Think of the **mean** as the physical *centre of mass* of your dataset. If you imagine placing your data points on a seesaw, the mean is the exact fulcrum point required to make the seesaw balance perfectly flat.

In our router example, four of the packets are clustered tightly together between 12 ms and 15 ms, but the 200 ms outlier is sitting incredibly far down the right-hand side of the seesaw. To prevent the seesaw from tipping over, the fulcrum (the mean) must be dragged drastically to the right—all the way to 50.6 ms. In statistics, this dragging effect is called **skew**.

The **median**, on the other hand, does not care about physical distance or mass; it only cares about *rank order*. Once the data is sorted, the median simply points to the item exactly in the middle of the queue.

Look at our sorted data again: 12, 12, **14**, 15, outlier. Whether that final outlier is 20 ms, 200 ms, or 2,000,000 ms, the middle packet will always be 14 ms. This mathematical property makes the median incredibly **robust**. In real-world engineering, where sensor glitches, network buffer spikes, or manufacturing anomalies occasionally produce wildly extreme numbers, the median protects your analysis from being ruined by a single bad data point.

1.5 Python to the Rescue

1.5.1 Setting up VS code for python

In this course we will show how to do large calculations with Python. To avoid the dreaded ‘it works on my machine!’ excuse and completely eliminate version conflicts, we are strictly aligning our Python tools with your **EEEN11202** module.

1.5.1.1 The Engineering Stack

- **Editor:** Visual Studio Code (VSCode) *VS Code is a lightweight, wildly popular code editor. Unlike heavy IDEs, it is essentially a highly customisable text editor that becomes powerful through extensions. Download it directly from code.visualstudio.com.*
- **Environment:** Devcontainers (Isolated and reproducible) *A Devcontainer runs a tiny, invisible Linux machine inside your computer. It guarantees that every student—regardless of whether they have an M-series Mac or a Windows PC—is running the exact same version of Python with the exact same settings.*
- **Package Manager:** uv (Blazing fast, modern standard) *Once inside your Devcontainer, you need to download statistical libraries. uv is a next-generation Python package manager written in Rust. It is blisteringly fast (often 10x to 100x faster than pip) and rigorously manages your project files.*

Getting Ready for Statistics: When you boot up your EEEN11202 devcontainer for our examples, you must initialise your project space and add the statistical libraries.

1. **Verify connection:** Look at the far bottom-left corner of the VS Code window for a coloured box containing a `><` icon. *If this box says ‘Dev Container’, you are safely inside the isolated Linux environment. If it does not, click the `><` icon and select ‘Reopen in Container’ from the drop-down menu before running any terminal commands!*
2. **Initialise:** Ensure you run `uv init` in the terminal first to create your configuration file.
3. **Install libraries:** Run the command below to download our specific tools.

```
1 # In your VSCode terminal inside the devcontainer:
2 uv add numpy scipy matplotlib
```

Listing 1: Setup

No PyCharm, no Anaconda, no conflicts. Just engineering efficiency!

In modern engineering, we rarely calculate statistics by hand. We leverage the power of the **SciPy Stack**, specifically `numpy` and `scipy.stats`.

1.5.2 Context: The ‘SciPy Stack’

One of the most confusing aspects for beginners is the terminology around ‘SciPy’.

- **SciPy (The Library):** A specific package containing algorithms.
- **SciPy (The Stack):** The entire ecosystem of tools that work together.

In Engineering, we treat the ‘SciPy Stack’ as a single cohesive toolkit, similar to how MATLAB operates. It is built as a hierarchy of layers, where each tool relies on the one below it.

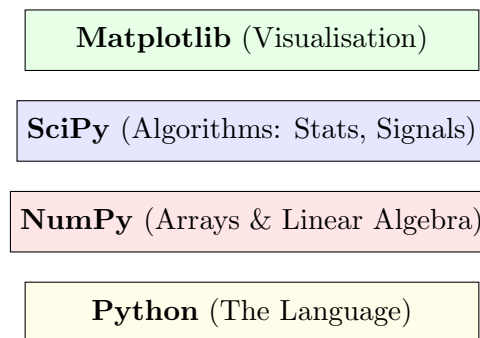


Figure 1: The Hierarchy of the Scientific Python Ecosystem. You cannot use the top layers without the bottom ones.

1.5.3 The Key Components

- **1. NumPy (Numerical Python):** This is the foundation. Python’s built-in lists are flexible but slow. NumPy introduces the `ndarray` (n-dimensional array), which is memory-efficient and extremely fast. *If you are doing math in Python, you are likely using NumPy.*
- **2. SciPy (Scientific Python):** This is the ‘Engine Room’. It assumes you already have your data in NumPy arrays. It provides the complex algorithms you would find in an engineering textbook:
 - `scipy.stats`: Probability distributions and tests.
 - `scipy.signal`: Filter design and waveforms.
 - `scipy.optimize`: Curve fitting and minimisation.
- **3. Matplotlib:** The standard library for plotting graphs. It is designed to work natively with NumPy arrays.
- **4. Pandas:** (Optional but common) Used for ‘Tabular Data’ (like Excel sheets or CSV files). It sits on top of NumPy but provides named columns and easier data cleaning tools.

NumPy (The Foundation)

- **Efficiency:** Highly optimised for N -dimensional arrays.
- **Speed:** Vectorised operations outperform standard Python loops.
- **Linear Algebra:** Essential for handling large datasets.

SciPy (The Toolbox)

- **scipy.stats:** Specialised functions for probability distributions (Normal, t -tests, p -values).
- **Fitting:** Tools to fit experimental data to mathematical models.
- **Integration:** Built-in solvers for complex engineering calculus.

```
1 import numpy as np
2 from scipy import stats
3 # Our dataset (e.g., latency in ms)
4 data = [12, 15, 12, 200, 14]
5 # Calculate Measures
6 mean_val = np.mean(data) # Output: 50.6
7 median_val = np.median(data) # Output: 14.0
8 mode_val = stats.mode(data, keepdims=True)
9 print(f"Mean: {mean_val}")
10 print(f"Median: {median_val}")
11 # Access the first element [0] to get the value, not the count
12 print(f"Mode: {mode_val.mode[0]}")
```

Listing 2: Python Code Example: Mean, mode, medium

Code Breakdown: Calculating Averages in Python

Below is a detailed explanation of the script.

- **import numpy as np:** This imports the **NumPy** library, which is the industry standard for engineering mathematics. We alias it as **np** to save typing. It contains highly optimised code for handling arrays and matrices.
- **from scipy import stats:** While NumPy handles basic math, **SciPy** (Scientific Python) contains more advanced statistical tools. We need this specifically for calculating the *Mode*, which NumPy does not support directly.
- **data = [...]:** We define our dataset as a standard Python list. Note that NumPy functions are smart enough to accept a list and convert it to an optimised array automatically.
- **np.mean(data) & np.median(data):** These functions calculate the location measures. They handle floating-point math automatically. If the list had an even number of elements, **np.median** would automatically interpolate the middle two values.
- **stats.mode(data, keepdims=True):** This is the most complex line.
 - **keepdims=True:** This ensures the output shape matches the input, preventing warnings in newer versions of SciPy.
 - **The Return Value:** This function does *not* return a single number. It returns a **ModeResult** object containing two arrays: the mode(s) and the count(s).

- `print(f"...")`: We use **f-strings** (formatted string literals) to insert variables directly into printed text.
- `mode_result.mode[0]`: Since `mode_result` is an object containing arrays, we cannot just print it. We access the `.mode` attribute (which is an array) and take the first element `[0]` to get the actual scalar value (e.g., 12).

1.5.4 Our Interactive Mean versus Median Demo

Now we know a little bit about Python, we can write a simple script to experiment with outliers and illustrate the issues with skew.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.widgets import Slider, Button
4
5 # 1. Initial Data
6 initial_data = [12, 12, 14, 15, 200]
7 num_points = len(initial_data)
8
9 # Calculate initial statistics
10 initial_mean = np.mean(initial_data)
11 initial_median = np.median(initial_data)
12
13 # 2. Set up the figure and adjust the layout to make room for 5 sliders
14 fig, ax = plt.subplots(figsize=(10, 6))
15 # Leave the bottom 45% of the window empty for the sliders
16 plt.subplots_adjust(bottom=0.45)
17
18 # 3. Plot the initial data
19 points_plot, = ax.plot(initial_data, np.zeros_like(initial_data), 'ko',
20                        markersize=12, zorder=3)
21
22 # Plot the Mean (Red Dashed) and Median (Blue Solid)
23 mean_line = ax.axvline(initial_mean, color='red', linestyle='--', linewidth
24                        =2.5, zorder=2)
25 median_line = ax.axvline(initial_median, color='blue', linestyle='-', linewidth
26                          =2.5, zorder=2)
27
28 # Text annotations
29 mean_text = ax.text(initial_mean, 0.05, f'Mean: {initial_mean:.1f}', color='red',
30                    ha='center', fontweight='bold')
31 median_text = ax.text(initial_median, -0.05, f'Median: {initial_median:.1f}',
32                      color='blue', ha='center', fontweight='bold')
33
34 # Format the axis to look like a 1D seesaw
35 ax.set_xlim(0, 250)
36 ax.set_ylim(-0.2, 0.2)
37 ax.set_yticks([])
38 ax.spines['top'].set_visible(False)
39 ax.spines['right'].set_visible(False)
40 ax.spines['left'].set_visible(False)
41 ax.spines['bottom'].set_position('zero')
42 ax.set_title("Interactive: Mean vs. Median", pad=20, fontsize=14, fontweight='bold')
43
44 # 4. Create the 5 Interactive Sliders
45 sliders = []
46 slider_axes = []
47
48 # Loop to create a vertically stacked slider for each data point
49 for i in range(num_points):

```

```

45     # Calculate vertical position for each slider (stacking them from bottom up
46     )
47     y_pos = 0.35 - (i * 0.07)
48     ax_slider = plt.axes([0.15, y_pos, 0.7, 0.03])
49
50     slider = Slider(
51         ax=ax_slider,
52         label=f'Packet {i+1} (ms)',
53         valmin=0.0,
54         valmax=250.0,
55         valinit=initial_data[i],
56         color='gray'
57     )
58     sliders.append(slider)
59     slider_axes.append(ax_slider)
60
61 # 5. The Update Function
62 def update(val):
63     # Read the current value of all 5 sliders
64     new_data = [slider.val for slider in sliders]
65
66     # Recalculate stats
67     new_mean = np.mean(new_data)
68     new_median = np.median(new_data)
69
70     # Update points and lines
71     points_plot.set_xdata(new_data)
72     mean_line.set_xdata([new_mean, new_mean])
73     median_line.set_xdata([new_median, new_median])
74
75     # Update text positions and values
76     mean_text.set_position((new_mean, 0.05))
77     mean_text.set_text(f'Mean: {new_mean:.1f}')
78
79     median_text.set_position((new_median, -0.05))
80     median_text.set_text(f'Median: {new_median:.1f}')
81
82     fig.canvas.draw_idle()
83
84 # 6. Link all sliders to the update function
85 for slider in sliders:
86     slider.on_changed(update)
87
88 # 7. Add a Reset Button (Optional but very helpful!)
89 ax_reset = plt.axes([0.8, 0.02, 0.1, 0.04])
90 btn_reset = Button(ax_reset, 'Reset', hovercolor='0.975')
91
92 def reset(event):
93     for slider in sliders:
94         slider.reset()
95 btn_reset.on_clicked(reset)
96
97 plt.show()

```

Listing 3: Mean versus Median Simulator

This code gives you an interactive set of sliders to change the latency of 5 data packets and instantly recalculates and plots both the mean and the median.

1.6 Summary so far

- **Data Types:** Identify if your variables are Discrete (counts) or Continuous (measurements).
- **Notation:** $\bar{x}$ for Sample Mean, μ for Population Mean.
- **Central Tendency:**
 - **Mean:** Mathematical centre, sensitive to outliers.
 - **Median:** Positional centre, robust.
 - **Mode:** Most frequent, good for categories.

Next: We will look at *Measures of Dispersion* (Range, Variance, Standard Deviation) to understand the “spread” of the data.

1.7 Measures of Dispersion (Spread)

So far in the previous section, we discussed how to measure the ‘average’ of a dataset, but often this is not enough information and we also have to consider the ‘spread’ of the data.

1.7.1 Why Central Tendency isn’t enough

Example 7 (The ‘Average’ Weather). Consider two cities with the same Mean Yearly Temperature (20°C).

- **City A (Stable):** Ranges from 18°C to 22°C .
- **City B (Extreme):** Ranges from -10°C to 50°C .

Question: Would you build the same bridge or roads in both?

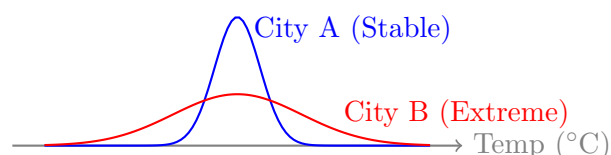


Figure 2: Frequency distribution of the highest daily temperatures.

Clearly we cannot use the same materials for each. To understand why dispersion dictates design, let’s look more carefully at the materials required for our two cities. Because City B has a huge temperature spread ($\Delta T = 60^{\circ}\text{C}$) compared to City A ($\Delta T = 4^{\circ}\text{C}$), the physical engineering requirements change drastically:

- **The Road Surface (Asphalt/Bitumen):** Asphalt is highly temperature-sensitive. In City A, you can use a cheap, standard bitumen binder because the temperature never leaves a comfortable ambient zone. If you used that same cheap asphalt in City B, it would become soft and deform under heavy truck tires at 50°C (rutting), and it would become glass-like and shatter under impact at -10°C . City B requires highly engineered, expensive Polymer-Modified Bitumen.

- **Thermal Expansion Joints:** Most metals expand when heated. For a 100-metre steel bridge in City A, the steel will only expand by about 5 millimetres over the year. You can practically ignore it. In City B, that same steel span will expand and contract by over 70 millimetres. If you don't install massive, expensive mechanical expansion joints, the bridge will literally tear itself off its concrete foundations during the summer.
- **Concrete and Moisture (Freeze-Thaw):** In City A, standard concrete is fine. In City B, the temperature drops below freezing. Any rainwater that seeps into micro-cracks in the concrete will freeze, expand, and blow the surface off the concrete (spalling). City B absolutely requires 'air-entrained' concrete—a special mix containing millions of microscopic air bubbles to give the freezing ice room to expand.

In short: the *mean* temperature tells you where to build. The *dispersion* (variance) tells you how much the materials will cost.

1.7.2 Why do we measure Spread?

In engineering design, the average is often a completely useless metric for safety. We must design for the extremes.

1.7.2.1 Real-World Engineering Constraints

Let's consider a few actual examples:

- A structural beam fails at the **maximum** load, not the average load.
- A circuit overheats at the **peak** current, not the average current.

The dispersion of our data gives us insights into the reliability and volatility of the system. If we want a highly reliable, low-risk system, we should minimise the dispersion. Of course, we can do this if we are designing a new component (or at least try!), but this is much harder to do with external variables like the weather.

Just as there are different measures of 'average', there are several measures of dispersion as well, each with its own pros and cons. Here we will focus on three commonly used measures of dispersion:

1. Range
2. Interquartile Range (IQR)
3. Variance and Standard Deviation

1.7.3 The Range

The simplest measure of spread.

$$\text{Range} = x_{\max} - x_{\min}$$

The range is the most intuitive measure of dispersion, but it is also the most fragile. Because the calculation strictly relies on the absolute maximum and minimum values, a single sensor glitch or anomaly will completely destroy the metric.

Example 8 (Revisiting Network Latency). Recall our network router example from the previous section, where we measured five packet latencies:

12ms, 12ms, 14ms, 15ms, **200ms**

If we calculate the range of this dataset:

$$\text{Range} = 200\text{ms} - 12\text{ms} = \mathbf{188\ ms}$$

The Engineering Trap: If you report a spread of 188ms to your project manager, they will assume the network is incredibly volatile and unpredictable. However, we know by looking at the data that 80% of the packets arrived within a tiny, highly stable 3ms window (between 12ms and 15ms).

The range completely ignores the tight internal clustering of the data. It allows one anomalous 200ms packet to hijack the entire narrative.

While the range is useful for a quick ‘sanity check’ of your dataset boundaries, it is rarely used for serious engineering analysis. We need statistical tools that account for *every* data point, not just the extremes. This leads us to Variance and Standard Deviation.

1.7.4 Interquartile Range (IQR)

If the Median is our robust ‘average’, the IQR is our robust ‘spread’. It measures the range of the **middle 50%** of your data, completely ignoring the extremes.

Definition: Quartiles

To find the IQR, we split the sorted data into four equal quarters:

- **Q_1 (Lower Quartile):** The 25th percentile.
- **Q_2 (Median):** The 50th percentile.
- **Q_3 (Upper Quartile):** The 75th percentile.

1.7.4.1 The IQR Formula

$$\text{IQR} = Q_3 - Q_1$$

Why the IQR is an Engineering Shield Standard Deviation and Range are easily destroyed by a single bad data point. Because the IQR explicitly throws away the highest and lowest quarters of your dataset before doing any math, it acts as a protective shield against anomalies.

Example 9 (Calculating the IQR). Let’s look at 7 temperature readings from a thermistor (in °C), sorted in order. One reading is clearly a system error:

10, 12, 14, **15**, 17, 18, **200**

1. **Find the Median (Q_2):** The middle value is clearly 15°C.
2. **Find Q_1 :** Look at the lower half of the data (10, 12, 14). The median of this lower half is 12°C.
3. **Find Q_3 :** Look at the upper half of the data (17, 18, 200). The median of this upper half is 18°C.

Now, calculate the spreads:

- **The Range:** $200 - 10 = \mathbf{190^\circ\text{C}}$ (*Falsely implies massive volatility*)
- **The IQR:** $18 - 12 = \mathbf{6^\circ\text{C}}$ (*Accurately describes the typical behavior*)

The IQR correctly tells the engineer that the vast majority of the time, this system is highly stable, fluctuating within a tight 6°C window.

1.7.4.2 Visualising the IQR

If you ever plot a **Box Plot** (or Box-and-Whisker plot) in Python using `matplotlib.pyplot.boxplot()`, the ‘box’ in the middle is drawn exactly from Q_1 to Q_3 . The width of that box is your IQR!

1.7.5 The Variance

How do we measure ‘average spread’ using *all* data points? We need some measure of distance the data is from the mean hence we can think of the average square distance from the mean i.e.

1. Find the distance of every point from the Mean: $(x_i - \bar{x})$.
2. Square them to remove negatives: $(x_i - \bar{x})^2$.
3. Average these squared distances.

Why Square? If we just added the distances $(x - \bar{x})$, the negatives would cancel the positives, summing to zero. Squaring makes everything positive.

1.7.5.1 Population vs Sample

This is the most common mistake in engineering statistics.

Population Variance (σ^2) Used when you have **all** the data (e.g., a census).

$$\sigma^2 = \frac{\sum (x - \mu)^2}{N}$$

Sample Variance (s^2) Used when you have a **subset** (e.g., 10 sensors out of 1000).

$$s^2 = \frac{\sum (x - \bar{x})^2}{n - 1}$$

1.7.5.2 Bessel’s Correction ($n - 1$)

Why do we divide by $n - 1$ for samples? Imagine you measure the height of 5 random people. It is highly unlikely you picked the shortest person on Earth AND the tallest person on Earth. Therefore, your sample spread is likely *smaller* than the true human population spread. Dividing by a smaller number ($n - 1$ instead of n) makes the result slightly *bigger*, correcting for this bias. As n gets huge (e.g., 1 million), difference between n and $n - 1$ becomes negligible.

1.7.5.3 Standard Deviation

The Problem with Variance: If we measure length in meters (m), Variance is in meters-squared (m^2). This is physically confusing.

In electrical engineering, if you are measuring the output voltage of a power supply and the mean is 5V, the variance might be calculated as $0.04V^2$. While mathematically necessary for intermediate steps, ‘volts squared’ is not a helpful unit when you are trying to determine if a 5V logic gate is going to fry. We need a measure of spread that shares the exact same physical units as our original data and our mean.

The Solution: Take the square root to return to the original units.

$$s = \sqrt{s^2} = \sqrt{\frac{\sum (x - \bar{x})^2}{n - 1}}$$

- **High SD:** Data is spread out (Noisy, Unreliable).
- **Low SD:** Data is clustered near the mean (Precise, Consistent).

The Engineering Reality: Tolerances and Quality Control Standard Deviation is arguably the most important statistical metric in modern manufacturing. When you buy a $1k\Omega$ resistor with a $\pm 1\%$ tolerance, the manufacturer is relying on standard deviation to guarantee that rating.

Later in the course, we will explore the **Normal Distribution** (the Bell Curve). When data follows this natural curve, Standard Deviation acts as a physical ruler for your dataset:

- Roughly **68%** of your manufactured components will fall within **one** standard deviation ($\pm 1s$) of the mean.
- Roughly **95%** will fall within **two** standard deviations ($\pm 2s$).
- Roughly **99.7%** will fall within **three** standard deviations ($\pm 3s$).

If a manufacturing process has a high standard deviation, the bell curve is wide and flat. This means a high percentage of your components will fall outside your acceptable safety limits and must be thrown in the bin. *Minimising standard deviation is how technology companies save millions of pounds in wasted silicon.*

Recipe (Calculating Variance (s^2) & Standard Deviation (s))

1. Calculate the **Mean** ($\bar{x}$) of the dataset.
2. Find the **deviation** for each data point by subtracting the mean: $(x_i - \bar{x})$.
3. **Square** each deviation to make it positive: $(x_i - \bar{x})^2$.
4. **Sum** all the squared deviations to get the ‘Sum of Squares’ (SS).
5. Divide the sum by $n - 1$ to find the **Sample Variance** (s^2).
 - *Note: Dividing by $n - 1$ (Bessel’s Correction) is standard for engineering samples.*
6. Take the **square root** of the variance to find the **Standard Deviation** (s).

1.7.6 Worked example

Example 10 (Calculating standard deviation and variance). Compute the standard deviation and variance of the following data. Data: $\{2, 4, 4, 4, 5, 5, 7, 9\}$. ($n = 8$).

Solution 11. *Step 1: Calculate Mean $\bar{x} = 40/8 = 5$.*

x	Step 2: Deviation $(x - \bar{x})$	Step 3: Squared $(x - \bar{x})^2$
2	-3	9
4	-1	1
...	...	...
9	+4	16
Sum (Step 4)	0	30

Step 5: $s^2 = \frac{30}{8-1} = \frac{30}{7} = 4.29$

Step 6: $s = \sqrt{4.29} \approx 2.07$

1.7.7 Python Implementation

Warning: NumPy calculates **Population** variance by default (divides by n). You must tell it to use the $n - 1$ correction.

```
1 import numpy as np
2
3 data = [2, 4, 4, 4, 5, 5, 7, 9]
4
5 # 1. Population SD (divides by n) - RARELY USED
6 pop_sd = np.std(data)
7
8 # 2. Sample SD (divides by n-1) - ENGINEERING STANDARD
9 # ddof = Delta Degrees of Freedom
10 sample_sd = np.std(data, ddof=1)
11
12 print(f"Sample Std Dev: {sample_sd:.2f}")
```

Listing 4: Calculating SD in Python

Note the parameter `ddof`:

- `ddof=0` (*NumPy Default*)
 - Calculates the **Population** Standard Deviation (σ).
 - Divides by N .
- `ddof=1`
 - Calculates the **Sample** Standard Deviation (s).
 - Divides by $n - 1$ (*Bessel's correction*).
- `ddof=2` (**and beyond**)
 - Divides by $n - 2$.
 - Used in more advanced statistics, such as calculating the standard error of a linear regression line.

1.7.8 Variance vs Standard Deviation

1.7.8.1 Variance (σ^2)

The Mathematical Tool

- **Units:** Squared (e.g., m^2 , s^2 , V^2).
- **Pro:** Mathematically clean. Variances can be added together directly.
- **Con:** Hard to interpret physically.

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$$

1.7.8.2 Standard Deviation (σ)

The Reporting Tool

- **Units:** Original (e.g., m , s , V).
- **Pro:** Intuitive. Describes the ‘average distance’ from the mean.
- **Con:** The math is messy (Square roots don’t add).

$$\sigma_{total} = \sqrt{\sigma_1^2 + \sigma_2^2}$$

Key Takeaway: Calculate with Variance, communicate with Standard Deviation.

1.8 Summary: Descriptive Statistics

We use statistics to distinguish the **Signal** (Location) from the **Noise** (Dispersion).

1.8.0.1 Location (The Signal)

- **Mean ($\bar{x}$):** The arithmetic centre. *Warning: Sensitive to outliers.*
- **Median:** The middle value. *Use for skewed data (e.g., Latency).*
- **Mode:** The most frequent value.

1.8.0.2 Dispersion (The Noise)

- **Variance (s^2) & Std Dev (s):** The math & reporting tools.
- **IQR:** The middle 50%. Highly robust.
- **Range:** Just $x_{max} - x_{min}$. Fragile.

The Golden Rule: Bessel’s Correction

When working with a **Sample** (which is almost always), you must divide by $n - 1$ to avoid under-estimating the spread.

$$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}}$$

1.9 Exercise Questions

Part A: The Idea

These questions test the mathematical definitions and calculations devoid of physical context.

Q1. Data Types

Classify the following abstract variables as Qualitative, Quantitative Discrete, or Quantitative Continuous:

1. The number of elements in a set.
2. A label assigned to a category (e.g., ‘Pass’, ‘Fail’).
3. A real number generated by a continuous mathematical function.

Q2. Measures of Central Tendency

Consider the following abstract dataset of values:

$$\{2, 3, 4, 4, 4, 4, 5, 10, 52\}$$

1. Calculate the sample mean ($\bar{x}$).
2. Determine the median and the mode.
3. If the value 52 was a data entry error and should have been 12, calculate the new mean and median. Which measure was more affected by the outlier?

Q3. Measures of Dispersion

Consider a small sample dataset:

$$\{2, 4, 6, 8, 10\}$$

1. Calculate the range of the data.
2. Calculate the sample variance (s^2). Be sure to explicitly show the use of Bessel's Correction.
3. Calculate the sample standard deviation (s).
4. If this data represented an entire population ($N = 5$) rather than a sample, what would the population variance (σ^2) be?

Part B: Exam style questions: Engineering Application

These questions test the exact same mathematical concepts as Part A, but applied to the Electrical Engineering scenarios discussed in the lecture.

Q4. Data Types in the Lab

Classify the following engineering measurements as Qualitative, Quantitative Discrete, or Quantitative Continuous:

1. The number of defective pixels on an LCD monitor.
2. The exact current (in mA) drawn by a microcontroller during sleep mode.
3. The colour band on a resistor (e.g., Red, Black, Orange).

Q5. Central Tendency & Network Latency

An engineer measures the round-trip latency (in ms) of 8 data packets sent to a remote server. The results are:

$$\{4 \text{ ms}, 5 \text{ ms}, 6 \text{ ms}, 6 \text{ ms}, 7 \text{ ms}, 9 \text{ ms}, 10 \text{ ms}, 42 \text{ ms}\}$$

1. Calculate the mean, median, and mode latency.
2. The 42 ms reading was caused by a temporary network routing spike. As an engineer summarising the “typical” performance of this network, would you report the mean or the median? Justify your choice based on the data distribution.

Q6. Component Tolerance & Spread

You pull 5 nominally identical $6\ \Omega$ resistors from a bin and measure their exact resistance. The readings are:

$$\{3\ \Omega, 4\ \Omega, 4\ \Omega, 5\ \Omega, 6\ \Omega\}$$

(Note: These are terrible resistors!)

1. Calculate the sample variance of this batch. What are the physical units of your answer?
2. Calculate the sample standard deviation. What are the physical units of this answer?
3. Referring to your answers regarding units, explain why engineers prefer to communicate uncertainty using Standard Deviation rather than Variance.
4. On the other hand mathematician prefer Variance, why?

Q7. Sampling & Bias

A factory produces 10,000 lithium-ion batteries per day. To test the reliability of the daily batch, the Quality Assurance team selects the first 50 batteries produced right after the machines are turned on at 8:00 AM. They find 0 defects and conclude the entire daily production is 100% reliable.

1. Is this a representative sample? Why or why not?
2. Identify a specific variable that might introduce bias into this sampling method.
(Hint: Think about machine temperature or calibration over a 24-hour shift).

Block 2

Probability Fundamentals

2.1 Outline of next block

Welcome to Block 2. In the previous block, we learned how to mathematically describe data that we have already collected (the past). Now, we shift our focus to predicting the likelihood of events before they happen (the future).

Probability forms the absolute mathematical foundation of risk assessment, reliability engineering, signal processing, and system design. In this block, we will establish the fundamental rules and notation that govern random events.

In this block we will introduce the following concepts:

- **Definitions & Notation**

- Sample Space (S), Events, Complement ($P(A') = 1 - P(A)$).

- **The Addition Law (Unions)**

- Mutually Exclusive: $P(A \cup B) = P(A) + P(B)$.
- General Rule: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

- **The Multiplication Law (Intersections)**

- Independent Events: $P(A \cap B) = P(A) \times P(B)$.
- Conditional Probability: $P(A \cap B) = P(A) \times P(B|A)$.

2.2 Probability Fundamentals

Probability forms the absolute mathematical foundation of risk assessment, reliability engineering, signal processing, and system design. In this part, we will establish the fundamental rules and notation that govern random events.

2.2.1 Definitions & Notation

Definition: Experiment and Sample Space

An **experiment** is any process whose outcome is subject to uncertainty.
The **sample space** (S) is the set of all possible outcomes.

We often consider probability experiments involving, flipping coins, rolling dice or drawing playing cards for illustration. However, In engineering, an experiment is a quality assurance test, a sensor reading, or a system failure check.

Example 12 (Testing Microcontrollers). Suppose we test 3 microcontrollers off a production line. Each either Passes (P) or Fails (F). The sample space has 8 possible outcomes:

$$S = \{PPP, PPF, PFP, PFF, FPP, FPF, FFP, FFF\}$$

Definition: Event

An **event** is any subset of the sample space.

If we define Event A as ‘exactly one microcontroller fails’, then:

$$A = \{PPF, PFP, FPP\}$$

Events are usually combined using standard set operations. Let A and B be events in sample space S .

- **Complement** ($\bar{A}$ or A'): All outcomes *not* in A .
 - $P(\bar{A}) = 1 - P(A)$
- **Intersection** ($A \cap B$): Outcomes in both A **AND** B .
- **Union** ($A \cup B$): Outcomes in A **OR** B (or both).
- **Mutually Exclusive** ($A \cap B = \emptyset$): Events that cannot happen at the same time. The intersection is the empty set ($\emptyset$).

Visualising Probability: Venn diagrams are the engineer’s best friend for probability. The outer box represents the entire sample space S (where total probability = 1). The circles represent events. If the circles overlap, the events can happen simultaneously (Intersection). If they don’t overlap, they are mutually exclusive.

2.2.2 The Addition Law

How do we calculate the probability of A **OR** B happening?

2.2.2.1 For Mutually Exclusive Events

If A and B cannot happen together ($A \cap B = \emptyset$), we simply add their probabilities:

$$P(A \cup B) = P(A) + P(B)$$

2.2.2.2 The General Addition Rule

If events *can* happen together, simply adding them will count the intersection twice. We must subtract the overlap:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Example 13 (Manufacturing Defects). A batch of printed circuit boards (PCBs) is tested.

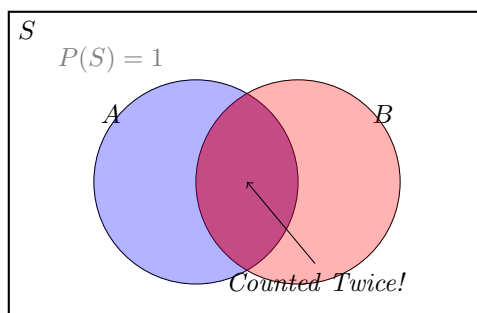
- Event A : The board has a soldering defect ($P(A) = 0.10$).
- Event B : The board has a trace defect ($P(B) = 0.05$).
- Both defects: $P(A \cap B) = 0.02$.

What is the probability a board has *at least one* defect ($A \cup B$)?

$$P(A \cup B) = 0.10 + 0.05 - 0.02 = \mathbf{0.13} \text{ (or 13\%)}$$

If we hadn't subtracted the 2% overlap, we would have mathematically rejected too many boards!

The addition law: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. Venn diagrams make the subtraction intuitive.



How to read this diagram:

- Start with the entire area of Event A : $P(A)$.
- Add the entire area of Event B : $+P(B)$.
- The intersection turns dark purple because we have counted it twice!
- To find the true Union area, we must subtract one instance of the intersection: $-P(A \cap B)$.

Example 14 (The 6-Sided Die). Consider rolling a fair 6-sided die. Let event A be rolling an even number, and let event B be rolling a number divisible by 3.

Questions:

1. Write down the elements of subsets A and B .
2. Calculate $P(A)$ and $P(B)$.
3. Write down the elements of the intersection (the event that the number is both even **and** divisible by 3) and calculate its probability, $P(A \cap B)$.
4. Calculate the probability that the number rolled is either even **or** divisible by 3, $P(A \cup B)$, by counting the elements.
5. Confirm the General Addition Law for this case.

6. Are events A and B mutually exclusive?

Solution 15. 1. Elements of A and B :

- Sample space $S = \{1, 2, 3, 4, 5, 6\}$
- A (Even numbers) $= \{2, 4, 6\}$
- B (Divisible by 3) $= \{3, 6\}$

2. Calculate $P(A)$ and $P(B)$:

- $P(A) = \frac{3}{6} = \frac{1}{2}$
- $P(B) = \frac{2}{6} = \frac{1}{3}$

3. Intersection $P(A \cap B)$:

- The only number both even **and** divisible by 3 is 6.
- $A \cap B = \{6\}$
- $P(A \cap B) = \frac{1}{6}$

4. Union $P(A \cup B)$ by counting:

- Elements in A **or** B (or both) are $\{2, 3, 4, 6\}$.
- There are 4 elements, so $P(A \cup B) = \frac{4}{6} = \frac{2}{3}$.

5. Confirming the Addition Law:

$$\begin{aligned} P(A \cup B) &= P(A) + P(B) - P(A \cap B) \\ &= \frac{3}{6} + \frac{2}{6} - \frac{1}{6} \\ &= \frac{4}{6} = \frac{2}{3} \quad (\text{Matches!}) \end{aligned}$$

6. Are they mutually exclusive?

- **No.** Mutually exclusive events cannot happen at the same time ($A \cap B = \emptyset$). Because rolling a 6 satisfies both conditions ($A \cap B = \{6\}$), they are not mutually exclusive.

2.2.3 The Multiplication Law

Often, knowing that one event has happened changes the probability of another event happening.

Definition: Conditional Probability

The probability of A given that B has already occurred is:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{for } P(B) > 0$$

The Engineering Meaning: Conditional probability shrinks your sample space. By stating ‘given that B has occurred’, we throw away the rest of the universe and make Event B

our new, smaller sample space. We then look at how much of A exists purely inside that new space.

Definition: Independent Events

Two events are **independent** if knowing one has occurred doesn't change the probability of the other.

$$P(A|B) = P(A) \quad \text{and} \quad P(B|A) = P(B)$$

2.2.3.1 The Multiplication Law (Intersections)

To find the probability of A **AND** B happening:

- **General Rule:** $P(A \cap B) = P(A) \times P(B|A)$
- **If Independent:** $P(A \cap B) = P(A) \times P(B)$

Example 16 (Why EEEs love Independence). Assume a critical server requires a cooling fan to prevent failure. A standard fan has a 10% chance of failing ($P(F_1) = 0.1$).

If we install two fans in parallel, and their failures are completely **independent** (e.g., they have separate power supplies), the probability of the server overheating is the probability they *both* fail:

$$P(F_1 \cap F_2) = P(F_1) \times P(F_2) = 0.1 \times 0.1 = \mathbf{0.01} \text{ (or 1\%)}$$

By exploiting the mathematics of independent intersection, engineers can build highly reliable systems out of cheap, unreliable parts.

Example 17 (Drawing Socks without Replacement). You have 4 red socks and 2 black socks in a basket. You draw two socks from the basket **without replacement**.

Let A be the event of drawing a red sock on your first attempt. Let B be the event of drawing a red sock on your second attempt.

Questions:

1. Calculate $P(A)$.
2. Assume event A occurred. Calculate the probability of drawing a second red sock, $P(B|A)$.
3. Calculate the probability of getting a red pair, $P(A \cap B)$.
4. Calculate the unconditional probability of drawing a red sock on the second attempt, $P(B)$. Is it the same as $P(B|A)$? (*Hint: What if the first sock was black?*)

Solution 18. 1. First Draw, $P(A)$:

- Total socks = 6. Red socks = 4.
- $P(A) = \frac{4}{6} = \frac{2}{3}$

2. Second Draw (Given A occurred), $P(B|A)$:

- Because we did not replace the first sock, the physical sample space has changed!
- Total socks left = 5. Red socks left = 3.
- $P(B|A) = \frac{3}{5}$

. **3. The Red Pair, $P(A \cap B)$:**

To find the probability of both events happening together, we use the **General Multiplication Law**.

Because the events are dependent, we multiply the probability of the first event by the conditional probability of the second:

$$P(A \cap B) = P(A) \times P(B|A)$$

- $P(A \cap B) = \frac{2}{3} \times \frac{3}{5} = \frac{6}{15}$

- Simplifying the fraction gives our final answer:

$$\frac{2}{5} \text{ (or 40\%)}$$

. **4. The Trap: Calculating Unconditional $P(B)$**

To find the total probability of the second sock being red, we must account for **both** possible paths.

- **Path 1:** First was Red **AND** Second is Red ($A \cap B$).
- **Path 2:** First was Black **AND** Second is Red ($A' \cap B$).

We already found Path 1: $P(A \cap B) = \frac{4}{6} \times \frac{3}{5} = \frac{12}{30}$. Now for Path 2: $P(A' \cap B) = \frac{2}{6} \times \frac{4}{5} = \frac{8}{30}$.

Add the two mutually exclusive paths together (Law of Total Probability):

$$P(B) = \frac{12}{30} + \frac{8}{30} = \frac{20}{30} = \frac{2}{3}$$

Solution 19. Comparing $P(B)$ and $P(B|A)$:

Let's look at the two numbers we just calculated:

- Unconditional probability of drawing a red sock second: $P(B) = \frac{2}{3}$
- Conditional probability (given the first was red): $P(B|A) = \frac{3}{5}$

The Core Definition of Dependence

Because $P(B) \neq P(B|A)$, the outcome of the first draw physically changes the probability of the second. Therefore, events A and B are **dependent**.

The Engineering Intuition: Notice that the unconditional $P(B)$ ($\frac{2}{3}$) is exactly the same as $P(A)$. If you draw a sock and do not look at it, the probability of the next sock being red is still the original ratio of the basket. The probability only collapses to $\frac{3}{5}$ when you actually observe the first event!

2.2.4 Summary

- **The Language of Probability:** All experiments have a sample space (S). Events are subsets of that space.
- **The Addition Law (Unions):** To find the probability of $A \cup B$, add their probabilities but remember to subtract the intersection!

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- **The Multiplication Law (Intersections):** To find the probability of $A \cap B$, multiply $P(A)$ by the conditional probability of B given A .

$$P(A \cap B) = P(A) \times P(B|A)$$

- **Independence:** If A happening doesn't change the probability of B happening, they are independent $\Rightarrow P(B|A) = P(B)$.

2.2.4.1 Next Up: Discrete Probability Distributions

Now that we know the basic rules of random events, we will map these events to numerical variables. Instead of just looking at 'Pass/Fail' events, we will learn how to calculate the probability of getting *exactly 3 failures* in a batch of 50.

2.3 Exercise Questions

Part A: The Idea

These questions test the mathematical definitions and calculations devoid of physical context.

Q1. Basic Probability & Set Operations

Let A and B be two events in a sample space S . You are given the following probabilities: $P(A) = 0.40$, $P(B) = 0.35$, and $P(A \cap B) = 0.15$.

1. Calculate the probability of A or B occurring, $P(A \cup B)$.
2. Calculate the conditional probability of B occurring given that A has occurred, $P(B|A)$.
3. Are events A and B mutually exclusive? Justify your answer using the provided numbers.
4. Are events A and B independent? Justify your answer mathematically.

Q2. Conditional Probability & Trees

An urn contains 5 white balls and 3 black balls. You draw two balls from the urn **without replacement**. Let W_1 be the event that the first ball is white, and B_2 be the event that the second ball is black.

1. Calculate $P(W_1)$.
2. Calculate $P(B_2|W_1)$.

3. Calculate the probability that the first ball is white **and** the second is black, $P(W_1 \cap B_2)$.
4. Calculate the unconditional probability that the second ball is black, $P(B_2)$, by considering both possible outcomes for the first draw. (*Hint: A tree diagram will help here*).

Part B: Exam Style Questions: Engineering Application

These questions test the exact same mathematical concepts as Part A, but applied to the Electrical Engineering scenarios discussed in the lecture.

Q3. The Addition Law & Manufacturing Defects

A batch of freshly manufactured printed circuit boards (PCBs) is undergoing visual inspection. The probability that a board has a soldering defect is 0.08. The probability it has a broken copper trace is 0.04. The probability that a board has **both** defects is 0.01.

1. What is the probability that a board has *at least one* of these defects?
2. If you randomly select a board and notice it has a broken copper trace, what is the probability that it also has a soldering defect?

Q4. The Law of Independence & System Redundancy

A critical data server requires a cooling fan to prevent thermal shutdown. A standard fan has a 5% chance of failing during a year of continuous operation ($P(F) = 0.05$). To improve reliability, an engineer installs two identical fans in parallel. They operate completely independently of one another.

1. Using the Multiplication Law for independent events, calculate the probability that **both** fans fail during the year.
2. The server will survive as long as *at least one* fan is working. Using the Complement rule, what is the probability that the server survives the year without thermal shutdown?
3. Based on these numbers, explain why EEEs heavily rely on the mathematics of independent events when designing critical systems.

Q5. The Problem of ‘Luck’ Resolved (Bridging the Gap)

In Module 1, we asked: “*You flip a fair coin 10 times. You should get 5 heads. You actually get 8. Is the coin biased, or is it just bad luck?*” Now we have the mathematical tools to solve this without any advanced formulas!

Assume the coin is perfectly fair ($P(H) = 0.5$, $P(T) = 0.5$) and each flip is completely independent.

1. Imagine one specific sequence of 10 flips: getting 8 heads in a row, followed by 2 tails ($H - H - H - H - H - H - H - H - T - T$). Using the Multiplication Law for independent events, calculate the exact probability of this specific sequence occurring.
2. Of course, $H - H - H - H - H - H - H - H - T - T$ is not the only way to get 8 heads. You could get $T - H - H - H - H - H - H - H - H - T$, etc. The mathematical formula for finding the number of possible combinations is $\frac{n!}{k!(n-k)!}$. Using $n = 10$ and $k = 8$, calculate how many different ways you can arrange 8 heads and 2 tails.

3. Because each of these combinations is **mutually exclusive**, we can use the Addition Law. Multiply your answer from Part 1 by your answer from Part 2 to find the total probability of getting exactly 8 heads.
4. Does this percentage prove the coin is mathematically biased, or does it prove that 8 heads can happen purely by chance?

Block 3

Discrete Probability Distributions

3.1 Introduction

In the previous block, we looked at the probability of specific, single events (like drawing a red sock). In engineering, we usually want to model a whole system of events mathematically. To do this, we use **Random Variables**.

Definition: Discrete Random Variable

A discrete random variable (usually denoted by a capital letter like X) is a function that assigns a **numerical value** to each outcome in a sample space. It can only take on a countable number of distinct values (e.g., 0, 1, 2, 3...).

For example, instead of writing out the sample space for testing three microcontrollers as $\{PPP, PPF, \dots, FFF\}$, we can simply define a random variable X as ‘the number of failed chips’. X can then take the discrete values $x = 0, 1, 2$, or 3.

Before we look at complex models, we must understand how to find the **average** outcome of a random process.

Definition: Expected Value $E(X)$

The Expected Value (or mean) of a discrete random variable is the weighted average of all possible outcomes:

$$E(X) = \mu = \sum_{i=1}^n x_i P(x_i)$$

Example 20 (Component Testing). A technician tests a batch of 3 sensors. The probability of x sensors being faulty is given below:

x	0	1	2	3
$P(X = x)$	0.70	0.20	0.08	0.02

Calculate the expected number of faulty sensors, $E(X)$.

Solution 21. To find $E(X)$, we multiply each outcome by its probability and sum them up:

$$\begin{aligned}
 E(X) &= (0 \times 0.70) + (1 \times 0.20) + (2 \times 0.08) + (3 \times 0.02) \\
 &= 0 + 0.20 + 0.16 + 0.06 \\
 &= \mathbf{0.42 \text{ sensors}}
 \end{aligned}$$

Engineering Interpretation: If we tested thousands of batches, we would find an average of 0.42 faults per batch.

3.2 The Binomial Distribution

The Binomial Distribution is the ultimate tool for Quality Assurance engineers. It models the number of **successes** in a fixed number of **independent trials**.

3.2.0.1 Conditions for a Binomial Model

To use this model, your experiment must meet four strict rules:

1. There is a fixed number of trials (n).
2. Each trial has only two outcomes ('Success' or 'Failure').
3. The probability of success (p) is the same for every trial.
4. The trials are completely **independent**.

Definition: The Binomial Formula

If $X \sim \text{Bin}(n, p)$, the probability of getting exactly k successes is:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Where $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ is the number of possible combinations.

Example 22 (Is the coin biased?). In Part 1, we asked: 'You flip a fair coin 10 times. You should get 5 heads. You actually get 8. Is the coin biased, or is it just bad luck?'

Now we have the exact formula to solve this. Assume the coin is perfectly fair ($p = 0.5$). Let X be the number of heads in $n = 10$ independent flips.

Question:

1. Calculate the exact probability of getting **exactly 8** heads by pure random chance.
2. **If** each person in a class of 300 students was to do this experiment, how many would get exactly 8 heads?

Solution 23. 1. Here, $X \sim \text{Bin}(10, 0.5)$. We want to find $P(X = 8)$.

Calculation:

$$\begin{aligned} P(X = 8) &= \binom{10}{8} (0.5)^8 (1 - 0.5)^{10-8} \\ &= \frac{10!}{8!(2)!} (0.5)^8 (0.5)^2 \\ &= 45 \times (0.5)^{10} \\ &= 45 \times 0.000976 \\ &= \mathbf{0.0439 \text{ (or } 4.39\%)} \end{aligned}$$

2. Expected number of students:

$$E(X) = 300 \times 0.0439 \approx \mathbf{13 \text{ students}}$$

. **Engineering Conclusion:**

- There is a 4.39% chance of getting exactly 8 heads from a perfectly fair coin just by luck.
- While 4.39% is low (about 1 in 23), it is **not impossible**.
- Out of 300 students, we actually expect 13 of you to see this result!
- Therefore, seeing 8 heads does not definitively prove the coin is mathematically rigged. It just means you witnessed a slightly rare event.

(Note: We will learn exactly where to draw the ‘impossible’ line when we cover Hypothesis Testing in Block 5!)

Example 24 (Manufacturing Yield). A factory produces LED bulbs. Historically, 5% of the bulbs are defective ($p = 0.05$). An engineer pulls a random sample of 10 bulbs off the assembly line ($n = 10$).

Questions:

1. What is the probability that **exactly 0** bulbs are defective? (A perfect batch).
2. What is the probability that **exactly 2** bulbs are defective?

Solution 25. Let X be the number of defective bulbs. $X \sim \text{Bin}(10, 0.05)$.

1. Exactly 0 defective ($k = 0$):

$$\begin{aligned} P(X = 0) &= \binom{10}{0} (0.05)^0 (1 - 0.05)^{10-0} \\ &= 1 \times 1 \times (0.95)^{10} \\ &= \mathbf{0.5987} \text{ (or 59.87\%)} \end{aligned}$$

2. Exactly 2 defective ($k = 2$):

$$\begin{aligned} P(X = 2) &= \binom{10}{2} (0.05)^2 (0.95)^8 \\ &= \frac{10!}{2!(8)!} \times 0.0025 \times 0.6634 \\ &= 45 \times 0.0025 \times 0.6634 \\ &= \mathbf{0.0746} \text{ (or 7.46\%)} \end{aligned}$$

Because the Binomial distribution is highly structured, calculating its expected value and spread doesn’t require building complex tables!

3.2.0.2 Binomial Shortcuts

If $X \sim \text{Bin}(n, p)$:

- **Mean (Expectation):** $\mu = E(X) = np$
- **Variance:** $\sigma^2 = np(1 - p)$
- **Standard Deviation:** $\sigma = \sqrt{np(1 - p)}$

Example 26 (Expected Failures). If we sample 10 bulbs with a 5% defect rate, how many do we *expect* to be defective?

$$E(X) = 10 \times 0.05 = \mathbf{0.5} \text{ bulbs}$$

Calculating factorials by hand is fine for $n = 10$, but not practical for $n = 10,000$. Engineers use the `scipy.stats` library to calculate complex probabilities instantly.

Solving the ‘8 Heads’ Problem: We want exactly 8 heads from 10 flips ($p = 0.5$). We use the Probability Mass Function (`pmf`).

```
1 from scipy.stats import binom
2
3 # binom.pmf(k, n, p)
4 p_8_heads = binom.pmf(8, 10, 0.5)
5
6 print(f"Probability of 8 heads: {p_8_heads:.4f}")
7 # Output: 0.0439
```

Listing 5: Our 8 heads python style

Notice that Python gives us the exact same 4.39% we calculated by hand!

When using the `scipy.stats` library to calculate discrete probabilities, the `pmf` (Probability Mass Function) command requires three core arguments to map exactly to our mathematical formula:

k = 8: The target number of ‘successes’ you want to evaluate (often denoted as x in our mathematical formulas). In this case, exactly 8 heads.

n = 10: The total number of independent trials. Here, we are flipping the coin 10 times.

p = 0.5: The probability of success on a single, isolated trial (a 50% chance of getting heads on any given flip).

If you ever need to find the cumulative probability—such as the probability of getting **8 or fewer** heads ($P(X \leq 8)$)—you simply swap `.pmf` for `.cdf` (Cumulative Distribution Function), leaving the three arguments exactly the same.

3.3 The Poisson Distribution

While the Binomial distribution counts successes in a *fixed number of trials*, the **Poisson Distribution** counts events that occur randomly over a **continuous interval** (like time, area, or volume).

3.3.0.1 When to use Poisson in EEE:

- Number of network packets lost per minute.
- Number of microscopic defects per square meter of silicon.
- Number of server crashes per month.

Definition: The Poisson Formula

If $X \sim \text{Pois}(\lambda)$, the probability of exactly k events occurring is:

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$$

Where λ (lambda) is the average rate of occurrence, and $e \approx 2.718$.

Example 27 (Fiber Optic Cable Faults). A long-haul fiber optic network experiences an average of $\lambda = 2$ dropped connections per day.

Questions:

1. What is the probability that the network experiences **exactly 0** drops tomorrow? (A perfect day).
2. What is the probability that the network experiences **at most 2** drops?

Solution 28. Let X be the number of dropped connections per day. $X \sim \text{Pois}(2)$.

1. Exactly 0 drops ($k = 0$):

$$\begin{aligned} P(X = 0) &= \frac{e^{-2} 2^0}{0!} \\ &= \frac{0.1353 \times 1}{1} \\ &= \mathbf{0.1353} \text{ (or 13.5\% chance of a perfect day)} \end{aligned}$$

. **2. At most 2 drops ($X \leq 2$):** This means 0 drops, 1 drop, **or** 2 drops. We must add the probabilities!

$$\begin{aligned} P(X \leq 2) &= P(X = 0) + P(X = 1) + P(X = 2) \\ &= e^{-2} \left(\frac{2^0}{0!} + \frac{2^1}{1!} + \frac{2^2}{2!} \right) \\ &= 0.1353 \times \left(1 + 2 + \frac{4}{2} \right) \\ &= 0.1353 \times 5 \\ &= \mathbf{0.6765} \text{ (or 67.65\%)} \end{aligned}$$

The Poisson distribution has a unique and beautiful mathematical property regarding its average and its spread.

3.3.0.2 Poisson Shortcuts

If $X \sim \text{Pois}(\lambda)$:

- **Mean (Expectation):** $\mu = E(X) = \lambda$
- **Variance:** $\sigma^2 = \lambda$

The Mathematical Oddity

In a Poisson distribution, the mean is **exactly equal** to the variance. As the average rate of events increases, the uncertainty (spread) around that rate increases at the exact same pace.

For the network faults example ($\lambda = 2$), we can calculate both exact and cumulative probabilities.

Solving the ‘At most 2 drops’ Problem: Instead of manually adding $P(X = 0) + P(X = 1) + P(X = 2)$, we can use the Cumulative Distribution Function (`cdf`) to add them automatically!

```
1 from scipy.stats import poisson
2
3 # 1. Exactly 0 drops (PMF)
4 p_0 = poisson.pmf(0, 2)
5 print(f"Exactly 0 drops: {p_0:.4f}")
6 # Output: 0.1353
7
8 # 2. At most 2 drops (CDF)
9 p_up_to_2 = poisson.cdf(2, 2)
10 print(f"At most 2 drops: {p_up_to_2:.4f}")
11 # Output: 0.6767
```

Listing 6: Poisson Example

When modelling the Poisson distribution using `scipy.stats`, the syntax is very similar to the Binomial setup, but the arguments change to reflect the Poisson parameters:

k: The specific number of events you are checking (e.g., 0 drops, or up to 2 drops). This corresponds to x in our mathematical formulas.

mu (λ): The expected average rate of occurrence. In this example, the historical average is 2 drops. *Note: SciPy uses the argument name **mu** instead of λ because **lambda** is a reserved mathematical keyword in the Python language!*

Which function do you use?

- `.pmf(0, 2)`: The **Probability Mass Function** calculates the probability of getting *exactly* $k = 0$ drops.
- `.cdf(2, 2)`: The **Cumulative Distribution Function** calculates the probability of getting *up to and including* $k = 2$ drops. It automatically does the heavy lifting of summing $P(X = 0) + P(X = 1) + P(X = 2)$.

Comment on Output Formatting: The `f"{p_0:.4f}"` syntax in the print statement is a Python f-string. The `:.4f` command tells Python to automatically round the output to exactly **4 decimal places** (floating-point numbers), which is the standard reporting format for probabilities.

3.4 Summary: Discrete Distributions

- **Random Variables (X)**
 - Maps sample spaces to numerical values.
 - **Expectation:** $E(X) = \sum xP(x)$ (The long-run average).
- **The Binomial Model**
 - Conditions: Fixed n , Binary outcomes (p/q), Independence.
 - Formula: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$
 - Mean $\mu = np$, Variance $\sigma^2 = np(1 - p)$.

- **The Poisson Distribution**

- Modelling rare events over a continuous interval (time/space).
- Formula: $P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$
- Mean and Variance are identical: $\mu = \sigma^2 = \lambda$.

3.4.0.1 Next Up: Continuous Probability Distributions

So far, we have only counted discrete ‘events’ (like 3 broken bulbs or 2 dropped packets). But what happens when our random variable is a continuous physical measurement, like the exact voltage of a battery or the thermal noise in a circuit? Next, we introduce the famous Normal Curve.

3.5 Exercise Questions

Part A: The Idea

These questions test the mathematical definitions and calculations devoid of physical context.

Q1. Discrete Random Variables & Expectation

A discrete random variable X has the following probability mass function (PMF):

x	0	1	2	3
$P(X = x)$	0.4	0.3	0.2	k

1. Find the value of k , knowing that the sum of all probabilities must equal 1.
2. Calculate the expected value, $E(X)$.

Q2. The Binomial Distribution

Let X be a binomial random variable modelled as $X \sim \text{Bin}(n = 5, p = 0.4)$.

1. Calculate the exact probability $P(X = 2)$.
2. Calculate the cumulative probability $P(X \leq 1)$. (*Hint: This means $X = 0$ or $X = 1$.*)
3. Calculate the mean (μ) and the variance (σ^2) of this distribution using the shortcut formulas.

Q3. The Poisson Distribution

Let X be a Poisson random variable modelled as $X \sim \text{Pois}(\lambda = 3)$.

1. Calculate the exact probability $P(X = 0)$.
2. Calculate the exact probability $P(X = 2)$.
3. What is the variance of this distribution?

Part B: Exam style questions: Engineering Application

These questions test the exact same mathematical concepts as Part A, but applied to the Electrical Engineering scenarios discussed in the lecture.

Q4. Expectation in Component Testing

A technician tests batches of 4 prototype relays. Let X represent the number of relays in a batch that fail the high-voltage stress test. Historical data gives the following distribution:

x	0	1	2	3	4
$P(X = x)$	0.65	0.20	0.10	0.04	0.01

1. Calculate the expected number of failures per batch.
2. If the company tests 1,000 batches this month, how many total failed relays should the procurement team expect to replace?

Q5. Binomial Quality Assurance (Manufacturing Yield)

A silicon wafer fabrication plant produces microchips. The manufacturing process has a known defect rate of 2% ($p = 0.02$). An engineer pulls a random sample of 15 chips for inspection.

1. What is the probability that **exactly 0** chips in the sample are defective? (A perfect sample).
2. What is the probability that **exactly 1** chip is defective?
3. The entire daily production run consists of 50,000 chips. Calculate the expected number of defective chips produced in a day, and the standard deviation of the defects.

Q6. Poisson Fault Modelling (Network Reliability)

A submarine fiber optic cable experiences tiny optical flaws along its length. These flaws occur randomly at an average rate of 1.5 flaws per kilometre ($\lambda = 1.5$).

1. Calculate the probability of finding **exactly 0** flaws in a 1 km section of cable.
2. Calculate the probability of finding **exactly 2** flaws in a 1 km section of cable.
3. **Exam Challenge:** Calculate the probability of finding **more than 1** flaw in a 1 km section. (*Hint: Use the complement rule: $P(X > 1) = 1 - P(X \leq 1)$*).

Q7. System Selection: Binomial vs. Poisson

As the lead systems engineer, you must decide which mathematical model to use for two different risk assessments. State whether you would use the **Binomial** or **Poisson** distribution for each, and justify your choice based on the rules learned in the lecture.

1. **Scenario A:** Counting the number of cosmic-ray induced ‘bit-flips’ in a server’s RAM module over a continuous 30-day period.
2. **Scenario B:** Testing 20 specific mechanical switches on a control panel, where each switch has a 5% chance of sticking. You want to find the probability of exactly 3 switches sticking.